CENTRO PAULA SOUZA

FACULDADE DE TECNOLOGIA DE FRANCA

“Dr. THOMAZ NOVELINO”

TECNOLOGIA EM DESENVOLVIMENTO DE SOFTWARE MULTIPLATAFORMA

Cauã Martins

Gabriel Ferreira

Guilherme Campos

Rodrigo Avelar

PARAFLOW

FRANCA/SP

2026

Sumário

1. Equipe
2. Problema
3. Justificativa
4. Objetivos
5. Solução proposta
6. Arquitetura
7. Tecnologias utilizadas
8. IoT: sensores, atuadores e comunicação

1. Equipe

Nome	Função no projeto
Guilherme Barbosa B. Campos	**Líder do projeto** · Internet das Coisas e Aplicações (sensores/atuadores, comunicação e dashboard)
Gabriel Henrique Ferreira	Laboratório de Desenvolvimento Web (API RESTful, OpenAPI, banco de dados e React)
Rodrigo Avelar Santos	Estatística (gráficos, indicadores e análise dos dados) · Documentação
Cauã Henrique Nascimento	Programação para Dispositivos Móveis I (consumo da API e visualização/processamento)

2. Problema

Em oficinas, almoxarifados e pequenas indústrias, a contagem e a classificação de parafusos por **bitola** e por **comprimento** ainda são feitas manualmente. Esse processo é:

- **Lento**: exige que uma pessoa conte e separe item por item.
- **Sujeito a erro humano**: confundir medidas parecidas, errar a contagem.
- **Sem registro digital**: não há histórico do que foi separado, quando e em que quantidade.

O resultado prático é a falta de visibilidade sobre quantas peças de cada tipo existem no momento, o que atrasa a produção quando uma bitola acaba sem aviso prévio.

Quem poderia utilizar a solução

- Pequenas indústrias e oficinas mecânicas que compram e consomem parafusos de diferentes tamanhos e bitolas.
- Almoxarifados que precisam manter controle de estoque de itens de fixação.
- Estudantes e makers que queiram automatizar processos simples de triagem de peças pequenas.

3. Justificativa

Atualmente, quem lida com grande quantidade de parafusos precisa contá-los e separá-los por bitola e comprimento de forma manual. Esse trabalho consome tempo, favorece erros de contagem e de classificação e não deixa histórico do que entrou e saiu do estoque, o que atrapalha o controle e a reposição dos itens.

Diante disso, o ParaFlow surge como uma solução que automatiza a contagem e a separação, registra os dados e os apresenta em dashboards, com alertas de estoque baixo, promovendo ganho de produtividade e rastreabilidade. O uso de IoT permite acompanhar o estoque remotamente, acionar um atuador automaticamente a partir de uma regra de negócio e controlar o sistema manualmente por um operador, mesmo sem estar fisicamente próximo da máquina.

Para reforçar a justificativa, incluam:

- Tempo gasto no processo manual (se possível, estimem ou meçam: "contar 500 parafusos leva X minutos"). [PREENCHER]
- Alinhamento com algum ODS da ONU (o manual sugere), por exemplo **ODS 9 – Indústria, Inovação e Infraestrutura**. [CONFIRMAR com o grupo]

4. Objetivos

Objetivo geral

Automatizar a separação física de parafusos por comprimento e por bitola, contar cada peça processada e apresentar o estoque atualizado em tempo real em dashboards web e mobile, eliminando a contagem manual e reduzindo o erro humano associado a ela.

Objetivos específicos

- Coletar dados de comprimento e peso das peças por meio de sensores (**simulados no Wokwi**).
- Classificar cada peça por comprimento (5, 10, 15 e 20 mm) e por bitola (M2, M3 ou M4).
- Enviar os dados para a nuvem via protocolo MQTT e disponibilizá-los em um dashboard.
- Disponibilizar uma API RESTful documentada em OpenAPI, com persistência em banco de dados.
- Controlar o separador por comando (liga/desliga) a partir do dashboard e do aplicativo.
- Alertar quando o estoque de uma bitola ficar abaixo do limite ou quando uma peça fora do padrão de calibração for detectada.
- Analisar estatisticamente os dados gerados (média, desvio padrão, histórico de comprimentos e bitolas, entre outros).

5. Solução proposta

O ParaFlow é um sistema IoT que identifica, conta e direciona parafusos de acordo com o comprimento e a bitola, e mostra o estoque atualizado em um dashboard. O funcionamento da parte IoT, prototipada na plataforma Wokwi, é:

1. **Comprimento:** quatro sensores indexados (simulados por pushbuttons) representam as faixas de saída do trilho classificador (5, 10, 15 e 20 mm) e indicam em qual faixa a peça foi detectada.
2. **Bitola:** um sensor de peso (simulado por um potenciômetro, no lugar de uma célula de carga) lê o peso da peça, usado para identificar a bitola (M2, M3 ou M4) dentro da faixa de comprimento já conhecida.
3. **Classificação e contagem:** o microcontrolador **ESP32** cruza comprimento e bitola, classifica a peça e incrementa o estoque da bitola correspondente.
4. **Atuador automático:** um servo motor gira para a posição correspondente à combinação comprimento × bitola, direcionando a peça ao compartimento correto (carrossel de saída).
5. **Atuador manual:** um módulo relé liga e desliga o separador a partir de um switch no dashboard.
6. **Alertas:** um LED vermelho e um buzzer sinalizam estoque abaixo do limite mínimo ou peça fora da calibração; um LED verde indica que o separador está ligado.

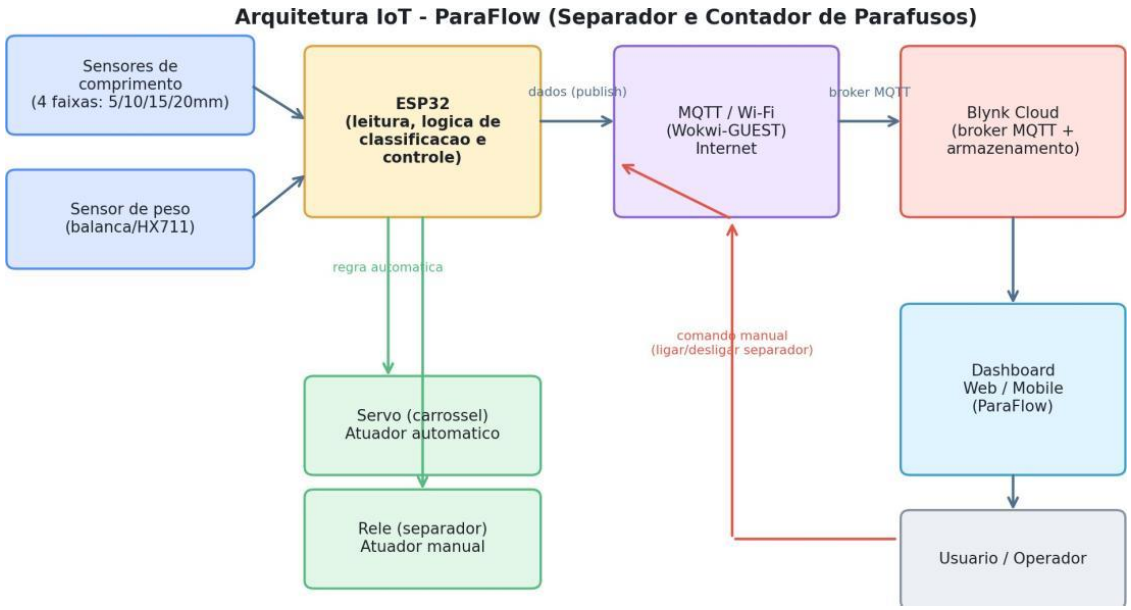
7. **Nuvem:** os dados são enviados via MQTT por Wi-Fi para o **Blynk IoT**, que apresenta o estoque, o peso, o comprimento, a bitola e o histórico, além de receber o comando manual do operador.

Os dados do projeto são **simulados**. A parte mecânica real (trilho com fenda progressiva e carrossel com vários compartimentos) não existe na biblioteca do Wokwi; por isso, sua função é representada eletronicamente pelos sensores indexados e por um único servo motor, cujo ângulo representa simbolicamente a posição do carrossel de destino.

Fluxo resumido: Sensores (comprimento e peso) → ESP32 (classificação e regras) → MQTT / Wi-Fi → Blynk → Dashboard → Usuário. O comando manual (liga/desliga) faz o caminho inverso, do dashboard até o ESP32, que aciona o relé do separador; o servo (carrossel) é acionado automaticamente pelo ESP32.

6. Arquitetura

A arquitetura representa o fluxo completo de dados: os sensores capturam as informações físicas, o ESP32 processa e decide, os dados sobem via MQTT/Wi-Fi até o Blynk Cloud, que os disponibiliza no dashboard para o usuário. O caminho inverso (em vermelho no diagrama) representa o comando manual enviado pelo dashboard para ligar ou desligar o separador, trafegando pelo mesmo canal MQTT de volta ao ESP32.



Legenda do fluxo: Sensores (comprimento e peso) → ESP32 (classificação e regras) → MQTT/Internet → Blynk Cloud → Dashboard → Usuário.

7. Tecnologias utilizadas

Camada	Tecnologia	Motivo da escolha
Microcontrolador	ESP32 DevKit	Wi-Fi integrado, ideal para IoT
Simulação	Wokwi	Permite prototipar o circuito sem hardware físico
Comunicação	MQTT sobre Wi-Fi	Protocolo leve, adequado para dados de sensores em tempo real
Plataforma IoT	Blynk IoT (broker MQTT, armazenamento e dashboard)	Dashboard web/mobile pronto, com pinos virtuais mapeados a tópicos MQTT
Back-end / API	[PREENCHER: ex.: Node.js +	[PREENCHER]

Camada	Tecnologia	Motivo da escolha
	Express]	
Banco de dados	MongoDB	[PREENCHER]
Front-end web	React	Exigido pela disciplina-chave
Mobile	EXPO	[PREENCHER]
Estatística	[PREENCHER: ex.: Python, JavaScript]	[PREENCHER]
Documentação da API	OpenAPI / Swagger	Exigido pela disciplina-chave
Versionamento	Git + GitHub	Exigido pelo manual

8. IoT: sensores, atuadores e comunicação

O projeto foi prototipado na plataforma **Wokwi**, com envio de dados via **MQTT** para o dashboard **Blynk IoT**. Os dados são simulados.

Componentes utilizados

Componente	Função	Como é utilizado na solução
ESP32 DevKit	Microcontrolador principal	Lê os sensores, executa a lógica de classificação e a calibração estatística, controla os atuadores e envia/recebe dados via MQTT para o Blynk.
4x Pushbutton	Sensores de comprimento (simulados)	Cada botão representa a saída de uma faixa do trilho classificador (5, 10, 15 e 20 mm), indicando em qual faixa a peça foi detectada.
Potenciômetro	Sensor de peso (simulação da célula de carga)	Simula a leitura analógica da balança, usada para identificar a bitola (M2, M3 ou M4) dentro da faixa de comprimento já conhecida.
Servo Motor (SG90)	Atuador automático: carrossel de saída	Gira para a posição correspondente à combinação comprimento × bitola, direcionando a peça ao compartimento correto. Acionado automaticamente pela regra de negócio.
Módulo Relé	Atuador manual: liga/desliga o separador	Controlado pelo switch do dashboard Blynk (V7); interrompe ou retoma o processamento de peças conforme o comando do operador.
LED vermelho + resistor 220 Ω	Indicador de alerta	Acende quando o estoque de alguma bitola fica abaixo do limite mínimo ou quando uma peça fora da calibração é detectada.
LED verde + resistor 220 Ω	Indicador de status do separador	Acende quando o separador está ligado e apaga quando desligado, reforçando o estado do sistema localmente, além do indicador no dashboard.
Buzzer	Alerta sonoro	Emite um sinal sonoro curto sempre que uma peça fora dos padrões de calibração é detectada, reforçando o alerta visual do LED vermelho.

Observação: os elementos físicos de separação mecânica (trilho com fenda progressiva, carrossel com múltiplos compartimentos) descritos na concepção do projeto não possuem componente equivalente na biblioteca do Wokwi. Por isso, sua função é representada eletronicamente por sensores indexados (pushbuttons) e por um único servo motor, cujo ângulo representa simbolicamente a posição do carrossel de destino.

Sensores

Sensor	Tipo	O que mede
4x Pushbutton	Digital	Faixa de comprimento da peça (5, 10, 15 ou 20 mm)
Potenciômetro	Analógico	Peso da peça (g), usado para identificar a bitola

Atuadores

Atuador	Acionamento	Função
Servo Motor (SG90)	Automático (regra de negócio)	Direciona a peça ao compartimento correto (carrossel)
Módulo Relé	Manual (switch no dashboard)	Liga e desliga o separador

Protocolo de comunicação: tópicos MQTT

O Blynk mapeia cada pino virtual (datastream) a um tópico MQTT interno da plataforma. A tabela relaciona cada pino virtual usado no firmware, seu tópico e a finalidade no projeto.

Pino	Tópico MQTT	Finalidade
V0	ds/peso	Envio do peso lido na balança (gramas), usado para identificar a bitola.
V1	ds/comprimento	Envio do comprimento identificado pelo sensor indexado (5/10/15/20 mm).
V2	ds/bitola	Envio da bitola identificada (M2, M3 ou M4) após cruzamento com o comprimento.
V3	ds/estoque_m2	Quantidade acumulada de parafusos M2 processados.
V4	ds/estoque_m3	Quantidade acumulada de parafusos M3 processados.
V5	ds/estoque_m4	Quantidade acumulada de parafusos M4 processados.
V6	ds/alerta	Sinaliza estoque baixo (abaixo do limite) ou peça fora do padrão de calibração.
V7	downlink/ds/separador	Recebe do dashboard o comando manual para ligar/desligar o separador.
V8	ds/carrossel	Envio da posição angular do servo, refletindo o destino de cada peça classificada.

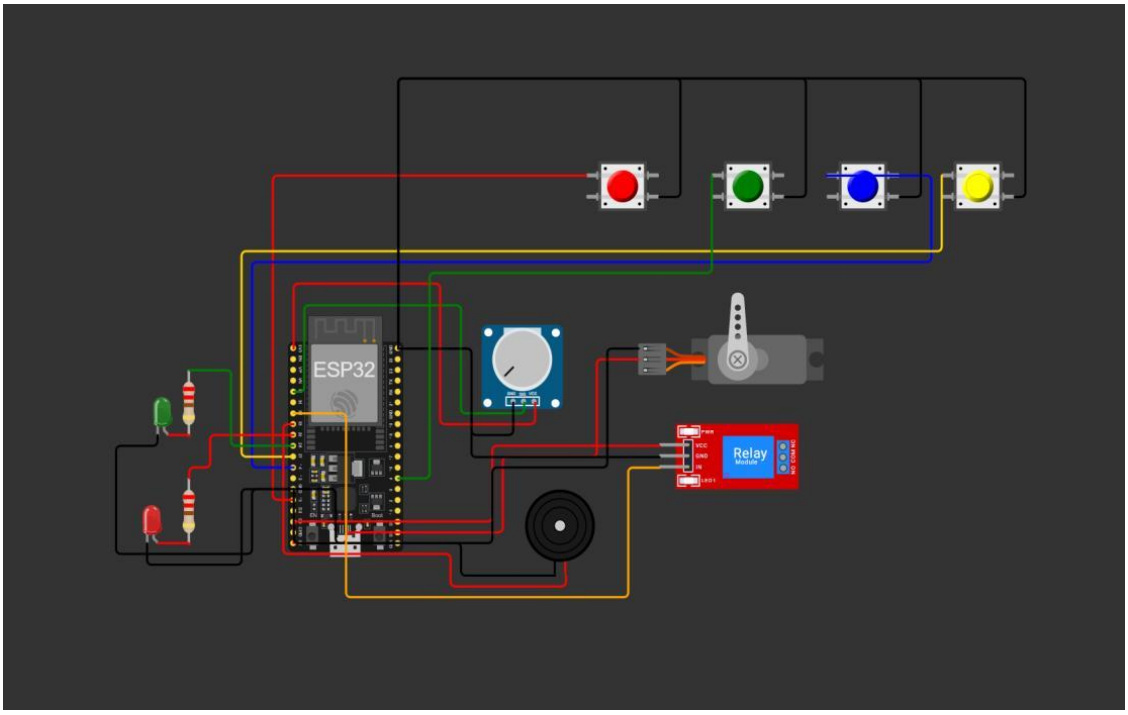
Os nomes dos tópicos seguem o padrão ds/<grandeza> para leitura (ESP32 → Blynk) e downlink/ds/<atuador> para comando (Blynk → ESP32).

Calibração estatística no firmware

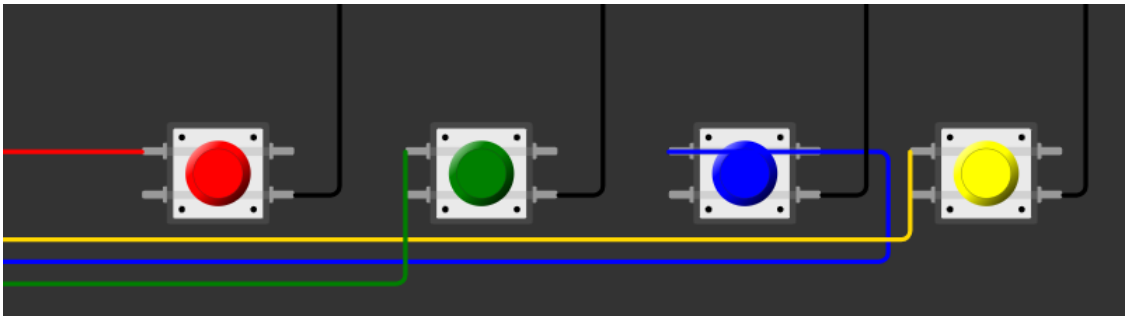
O firmware executa a lógica de classificação e a calibração estatística. Colocar faixas de peso usadas para separar M2, M3 e M4, limite mínimo de estoque que dispara o alerta e como a análise estatística ajusta a calibração

Esquema de ligação (prints do Wokwi)

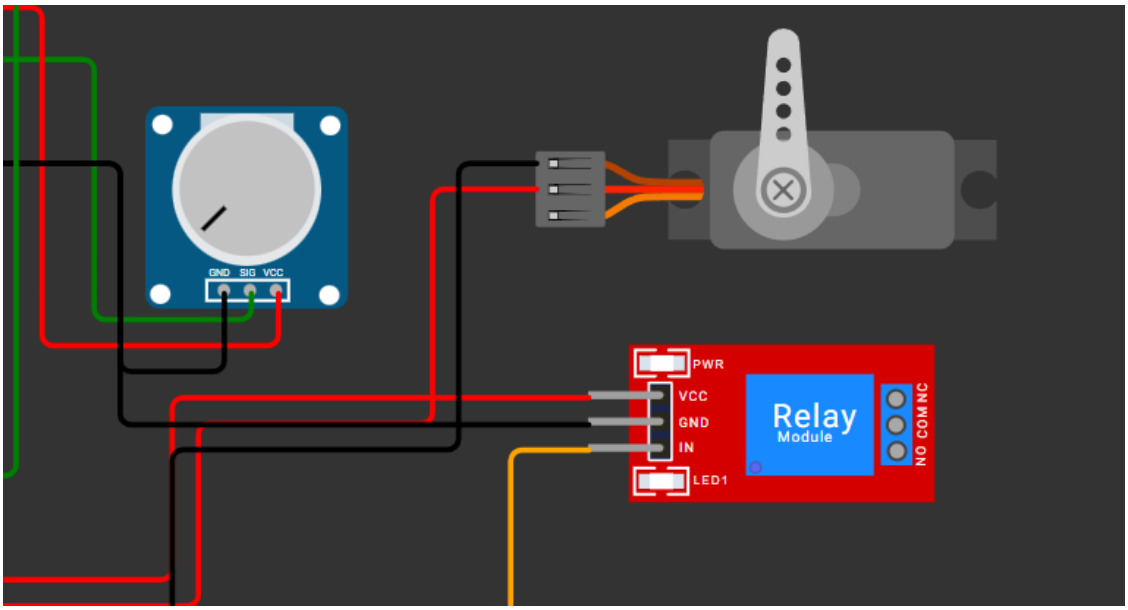
Visão geral do circuito



Detalhe dos sensores de comprimento (4 botões indexados)

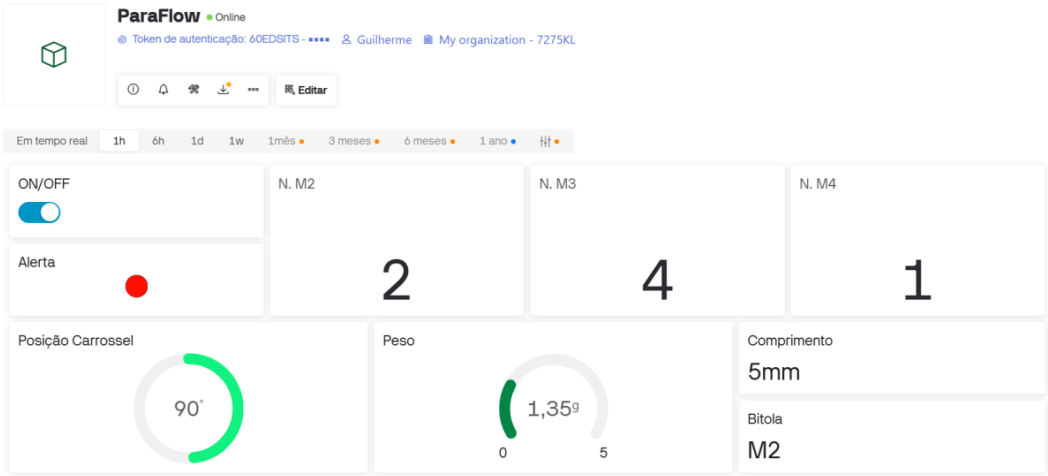


Detalhe da pesagem (potenciômetro), do servo (carrossel) e do relé (separador)

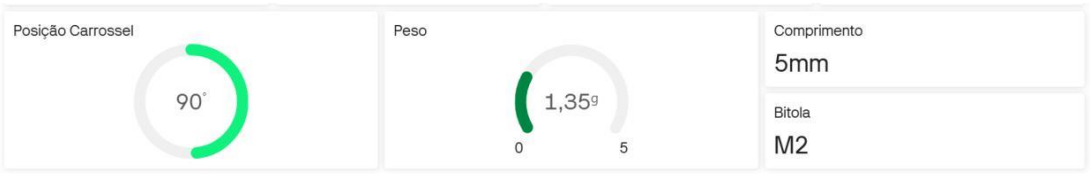


Dashboard Blynk IoT

Visão geral (estoque por bitola, peso, comprimento, bitola, posição do carrossel e alerta)



Switch de controle manual (liga/desliga o separador) e LED de alerta



COLOCAR: print do gráfico de histórico de estoque no Blynk, que ainda não foi incluído.